

LeHart Launch Readiness

What still stands between the current build and taking real money from a real customer — and whether starting again in .NET would help.

Reviewed against branch `claude/lehart-redesign`

UK direct-to-consumer, refurbished handsets

The short answer on .NET

Don't rewrite. A rewrite changes the language of the part that already works and doesn't touch the part that's missing.

Nearly everything standing between you and launch is a payment provider, a tax decision, a data-model change, or a legal obligation. None of it is caused by React or Firebase, and none of it gets easier in C#. You'd spend three to six months arriving back where you are now, minus the momentum.

There *is* a real technical change worth making — but it's targeted, not total. See [the fork](#).

The one case where .NET is the right call

Pick the language of whoever maintains this for the next three years. If that's you, and .NET is where you're fast and confident, then build the **commerce API** in .NET and let the existing React storefront call it. You don't have to choose between them — a storefront and a transactional backend are separate services that talk over HTTP.

Rewrite the front end only if you genuinely can't read or hire for it. Not because it feels less serious than a compiled language.

What's actually built

This matters more than the stack question, because it's the thing that decides your timeline. I went through the code rather than going from memory.

WORKING

Catalogue, search, accounts, admin

133 products in Firestore with a working search index, filtering, Firebase auth including Google sign-in, and an inventory console with image upload, stock editing and delete. Cart, wishlist and order records all persist. Trade-in quotes, coupons, delivery estimates and reviews have real endpoints.

DEMO ONLY

Payments — there is no payment processing

Checkout takes a card number in an ordinary text input and fabricates the last four digits. No processor, no authorisation, no money moves. It's a convincing walkthrough, not a till. **The raw card field must be deleted before this site is ever public** — accepting a card number into your own form puts you in a PCI scope you do not want and cannot satisfy.

DEMO ONLY

Returns, express pay, tax

The returns flow shows a three-step modal and then a success message — no record is created, nothing reaches an inbox. Apple Pay, Google Pay and PayPal buttons are visual. Every order is written with `tax: 0`.

DEMO ONLY

Newsletter

Signups write an email address into Firestore. There's no email platform behind it, no double opt-in, and no unsubscribe route — so as it stands you have a list you can't legally or practically send to.

STRUCTURAL

The inventory model is for new goods, not used ones

A product carries one grade, one battery health and a `stock` count. That describes a shelf of identical boxes. You sell serialised items: this specific handset, this IMEI, this battery health, bought for this much, warranty starting on this date. Until stock is per-unit you can't run the VAT margin scheme, can't answer "which handset did we send them", and can't price by real condition. **This is the deepest gap in the build, and it's a data-model problem — a rewrite in any language reproduces it unless you change the model.**

The fork that actually matters

Not React versus .NET. **Build the commerce core, or buy it.** Carts, tax, refunds, SCA and PCI scope are a solved commodity problem that is genuinely hard to get right and carries real liability when you get it wrong.

OPTION A

Keep it custom

Add Stripe, move orders to Postgres, build returns and refunds yourself. Total control and no platform fees, but every edge case, every compliance change and every 3am bug is yours.

Fits if the shop's whole appeal is something no platform can express, and you have developer time indefinitely.

OPTION B — RECOMMENDED

Buy the till, build the differentiator

Put the transactional core on Shopify. Payments, tax, refunds, PCI scope, staff admin and order management arrive solved on day one. Keep this React storefront in front of it via the Storefront API, or use it as the marketing site.

You then spend your build effort where you're actually different: grading, per-IMEI stock, trade-in, warranty.

Honest caveat: Shopify's stock model is SKU-based too, so serialised inventory still needs custom work — but that's a far smaller gap than building payments and refunds from nothing.

OPTION C

WooCommerce

Cheaper than Shopify and more open, at the cost of hosting, patching and security being your problem. A reasonable middle path if you already run WordPress; otherwise it trades a monthly fee for ongoing maintenance.

My recommendation is **B** unless you have a specific reason against it. The fastest safe route to taking real money is a system that already knows how to take money.

What you need to buy or build

Vendors below are picks, not a survey. Prices move — treat them as orientation and check current rates before committing.

AREA	PICK	WHY, AND WHAT TO WATCH
Payments	Stripe Adyen at real volume	UK entity, 3D Secure and SCA handled for you, Apple / Google Pay, Klarna and Clearpay resold through one integration, and a refunds API you'll lean on constantly. Using Stripe's hosted fields keeps you in the lightest PCI bracket. Watch: refurbished electronics is a high-chargeback category — expect underwriting questions and possibly a rolling reserve.
Email marketing	Brevo Klaviyo if flows matter more than cost	Brevo is already connected to this workspace, is EU-based which simplifies your GDPR paperwork, and covers marketing, transactional and SMS in one account. Klaviyo has better ecommerce segmentation and abandoned-basket flows for meaningfully more money.

AREA	PICK	WHY, AND WHAT TO WATCH
Transactional email	Brevo, or Postmark order, dispatch, refund receipts	Keep receipts separate from marketing in your own head even if one vendor sends both — a receipt that lands in spam becomes a support ticket. Postmark exists specifically because transactional deliverability is a different discipline. Either way: SPF, DKIM and DMARC on <code>lehart.co.uk</code> before your first send.
SMS	Twilio or Brevo SMS	Dispatch and delivery texts cut "where is my order" contacts sharply. Watch: UK networks now filter alphanumeric sender IDs that aren't registered on the UK Sender ID protection registry — register "LeHart" before you rely on it. Order updates and marketing texts also sit on different consent bases; don't blur them.
Phone number	An 03 number via Twilio or a business VoIP	03 numbers cost callers the same as a landline and count as basic rate, which is what consumer law expects for post-sale support. Avoid 084 and 087 entirely. Validate customer mobiles as <code>+44 7...</code> at entry — a mistyped number is a failed delivery.
Shipping	Royal Mail Click & Drop API DPD or Evri for tracked volume	Phones ship with lithium batteries installed, which is a declared dangerous-goods category with packaging rules and air restrictions. Your carrier account must be set up for it. Insure to value and use signed-for — high-value small parcels are the most stolen thing in the network.
Returns & refunds	Build on your order records plus a tracked returns label	The dedicated platforms (Loop, ReBOUND) mostly assume Shopify, which is another point for Option B. At your size the essentials are: a return record with a reason and a state, a prepaid tracked label, an inspection step, and a refund fired through the payment provider against the original charge.
Stolen-device checks	CheckMEND	Non-negotiable if you buy trade-ins. Selling a blocked or stolen handset costs you the sale, the stock and the reputation, and the checks are cheap per unit. Record the check result against the IMEI so you can prove you did it.
Accounting	Xero or QuickBooks check margin-scheme support first	Second-hand VAT is unusual enough that it's a genuine selection criterion, not a detail. Ask the question during the trial, not after your first quarter.
Fraud	Stripe Radar	Refurbished phones are among the most targeted categories online. Baseline: address verification on, extra scrutiny where billing and delivery differ on a first order, and record the dispatched IMEI so you can evidence exactly what you sent when a chargeback lands.
Support	Crisp or Freshdesk Gorgias if you go Shopify	One shared inbox with order context attached. The thing that scales badly is answering support from a personal email account, and it scales badly sooner than you'd expect.
Analytics	GA4 plus your payment data	Treat the payment provider as the source of truth for revenue and the analytics tool as the source of truth for behaviour. They will disagree; that's normal, and reconciling them to the penny is wasted effort.

UK obligations that bite

These are the ones that catch people running this exact kind of shop. Several change what the software has to store, which is why they belong in a build plan and not only in a folder at your accountant's.

The VAT margin scheme is the big one

For second-hand goods you can account for VAT on your *margin* rather than the full sale price, which on refurbished stock is a large cash difference. The conditions shape your database: you must keep a stock book recording purchase price and sale price **per individual item**, and your invoices must not show VAT as a separate line. That's the second independent reason your inventory has to be per-IMEI rather than a stock count.

Returns are two overlapping rights, not one

- **Distance selling.** Customers can cancel within 14 days of delivery for any reason at all, and you refund within 14 days of getting the goods back.
- **Faulty goods.** A separate short-term right to reject in the first 30 days, then repair or replacement — and for the first six months the burden is on you to show a fault wasn't there at delivery.

Refurbished stock generates far more faulty-goods claims than new stock, so build the returns flow around inspection and outcome, not just "money back".

Also on the list

- **Total price up front.** Recent consumer law bans drip pricing — mandatory charges have to be in the headline price, not revealed at checkout.
- **Reviews must be genuine.** Fake or incentivised-without-disclosure reviews are now explicitly prohibited, with real penalties.
- **Electrical and battery waste.** Selling electricals brings take-back obligations for old equipment and for batteries. Small, boring, enforced.
- **Data protection.** ICO registration and annual fee, a privacy notice that matches what you actually do, and a written processor agreement with each vendor above.
- **Email consent.** Marketing needs consent or a narrow existing-customer exemption; every send carries an unsubscribe; keep proof of when and how each address was collected. Your current Firestore list has none of this attached to it.
- **Insurance.** Product liability plus stock cover. Also check whether your council requires second-hand dealer registration — it varies.

Sequence

In dependency order — each phase is only sensible once the one above it is done. Durations assume one experienced developer and are rough.

01

BEFORE A SINGLE REAL ORDER · 4–8 WEEKS

Be able to take money, safely

- Delete the raw card field. First, before anything else.
- Real payments through a provider's hosted fields, with SCA.
- Decide build-versus-buy — everything downstream depends on it.
- Per-item inventory with IMEI, so stock, VAT and warranty all have something to hang on.
- An order that has real states, and a refund path that can reverse a real charge.
- Order and dispatch emails that actually send.
- Terms, returns policy and privacy notice that describe what the software does.

02

BEFORE VOLUME · 3–5 WEEKS

Be able to run the shop

- Returns that create a record with a reason, an inspection and an outcome.
- Shipping labels generated from the order, with tracking sent to the customer.
- IMEI recorded at dispatch — your evidence in every dispute.
- Stolen-device checks wired into trade-in intake.
- A shared support inbox.
- Margin-scheme stock book feeding your accounts.

03

ONCE ORDERS ARE STEADY · ONGOING

Grow demand

- Email platform with double opt-in, replacing the raw Firestore list.
- Abandoned basket, post-purchase and trade-in follow-up sequences.
- Dispatch and delivery SMS.
- Genuine review collection, after delivery.
- Analytics you actually read, and paid acquisition once the margin supports it.

How far this scales

Further than the business will need for a long time. Vercel and Firestore will serve thousands of orders a month without strain, and hosting will cost tens of pounds, not hundreds. The technical ceiling isn't what limits this shop.

What limits it is working capital to buy stock, capacity to process returns, someone to answer the phone, and reputation. Those are the constraints worth planning against.

Two things *do* break before the traffic does, and both are database-shaped rather than language-shaped:

- **Money doesn't belong in Firestore.** Orders, refunds, VAT records and stock ledgers want transactions, joins and ad-hoc queries you can hand an accountant. Firestore gives you none of those. This is the one genuine argument for changing technology — and the change is "put the transactional core on Postgres or a commerce platform", not "rewrite the site".
- **Reporting.** The first time someone asks "what was our average margin on Samsung stock last quarter" you will want SQL.

If you'd enjoy writing that transactional core in .NET, that's a perfectly good reason to write it in .NET. Just build it as a service the existing storefront calls, rather than as a reason to throw away work that already passes its tests.

BEFORE YOU ACT ON THIS

The tax, consumer-law and licensing points above are orientation — enough to know what to ask about, not a substitute for asking. Take the VAT margin scheme and your returns obligations to an accountant and a solicitor before launch, and confirm current thresholds, fees and provider rates directly with the source. They move, and I'd rather you check than take my word for a number.